Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет прикладної математики та інформатики
Кафедра математичного моделювання соціально-економічних процесів

Звіт

Індивідуальне завдання №1

Виконав:
студент групи Пма-42
Думанський Юрій
Перевірила:
доцент Лисецька О. Ю.

Львів-2026

Тема:

Об'єктно-орієнтоване програмування в Python: класи, інкапсуляція та статична типізація

Завдання:

Варіант 6.

Створити систему онлайн-замовлень супермаркету. Реалізувати класи Customer, Product, Order. Застосувати інкапсуляцію для номера телефону, написати декоратор для формування чеку, додати логування нестачі товарів та перевірку загальної кількості товарів (не більше 1000). Забезпечити анотування коду та тестування за допомогою mypy.

Хід роботи:

• Розробку програмного рішення виконано мовою Python з дотриманням парадигми ООП та використанням підказок типів. Було спроєктовано базові класи Product, Customer та Order для моделювання предметної області.

• Для захисту конфіденційних даних реалізовано механізм інкапсуляції: доступ до приватного атрибута номера телефону покупця налаштовано через гетери та сетери за допомогою декоратора @property. Також для всіх об'єктів перевизначено магічні методи str та repr для коректного відображення стану системних сутностей.

• Реалізовано бізнес-логіку обробки замовлень: застосовано @classmethod для валідації ліміту товарів у кошику та @staticmethod для агрегації складських залишків. Написано кастомний декоратор receipt_decorator для генерації фінального чека. Налаштовано механізм файлового логування (shortage_log.txt) для фіксації випадків нестачі асортименту. Фінальний код перевірено статичним аналізатором mypy та наведено в додатку 1.

Результати:

Програма коректно ініціалізує сутності та проводить транзакції: автоматично обчислюється 10% знижка на товари, дорожчі за середню ціну кошика, після чого генерується консольний касовий чек. Спроби замовлення товарів, яких бракує на складі, ігноруються системою з відповідним записом даних клієнта у лог-файл

shortage_log.txt. Перевірка коду через mypy не виявила конфліктів типів (скріншоти консольного виводу та вмісту лог-файлу наведено у додатках).

```
C:\Users\Юрій\Desktop\птіон\ind1>ind1.py
Загальна кількість товарів на складі: 77 шт.

Початковий телефон: +380501234567

==============================
         Касовий чек
==============================
Покупець: Іван Іваненко
Телефон: +380999876543
------------------------------
Хліб            | 60.00 грн
Молоко          | 45.00 грн
Кава            | 225.00 грн
------------------------------
Разом до сплати: 330.00 грн
==============================

Магічні методи (__str__ та __repr__):
Товар: Хліб, Ціна: 30.0 грн, В наявності: 48 шт.
Customer(customer_id=1, first_name='Іван', last_name='Іваненко', phone_number='+380999876543')
Замовлення для Іван Іваненко на суму 330.00 грн.
```

Рис. 1. Коснсольний вивід програми.

```
Success: no issues found in 1 source file
```

Рис. 2. Успішна перевірка коду модулем туру.

```
shortage_log.txt: Блокнот
Файл  Редагування  Формат  Вигляд  Довідка
Покупець: Іваненко Іван, Тел: +380999876543 | Нестача товару: Сир (Замовлено: 5, В наявності: 2)
```

Рис. 3. Файл shortage_list

Висновки:

У результаті виконання роботи було створено комплексну систему управління онлайн-замовленнями. На практиці закріплено інструментарій ООП у Python: роботу з інкапсуляцією, поліморфізмом (через перевизначення магічних методів), декораторами та файловою системою. Застосування анотацій типів у поєднанні з аналізатором mypy дозволило забезпечити високу надійність архітектури та уникнути помилок типізації на етапі розробки.

Додаток 1. Код програми.

```
import os
from typing import Callable, Any

# декоратор для видачі чеку
def receipt_decorator(func: Callable[..., Any]) -> Callable[..., Any]:
    """Декоратор, що формує та виводить чек після обробки замовлення."""
    def wrapper(self: Any, *args: Any, **kwargs: Any) -> Any:
        result = func(self, *args, **kwargs)
        if not self.ordered_items:
            print(f"[{self.customer.first_name} {self.customer.last_name}]
Замовлення порожнє або скасоване.")
            return result

        print("\n" + "="*30)
        print("Касовий чек".center(30))
        print("="*30)
        print(f"Покупець: {self.customer.first_name} {self.customer.last_name}")
        print(f"Телефон: {self.customer.phone_number}")
        print("-" * 30)
        for product, cost in self.ordered_items.items():
            print(f"{product.name:<15} | {cost:.2f} грн")
        print("-" * 30)
        print(f"Разом до сплати: {self.total_amount:.2f} грн")
        print("="*30 + "\n")
        return result
    return wrapper


class Product:
    def __init__(self, name: str, price: float, quantity_in_stock: int) -> None:
        self.name: str = name
        self.price: float = price
        self.quantity_in_stock: int = quantity_in_stock

    @staticmethod
    def calculate_total_stock(products: list['Product']) -> int:
        """Статичний метод для підрахунку загальної кількості товарів на складі."""
        return sum(p.quantity_in_stock for p in products)

    def __str__(self) -> str:
        return f"Товар: {self.name}, Ціна: {self.price} грн, В наявності:
{self.quantity_in_stock} шт."

    def __repr__(self) -> str:
        return f"Product(name='{self.name}', price={self.price},
quantity_in_stock={self.quantity_in_stock})"


class Customer:
    def __init__(self, customer_id: int, first_name: str, last_name: str,
phone_number: str) -> None:
```

```python
        self.customer_id: int = customer_id
        self.first_name: str = first_name
        self.last_name: str = last_name
        self.__phone_number: str = phone_number # Приватний атрибут
        self.cart: dict[Product, int] = {} # Словник товарів у кошику

    @property
    def phone_number(self) -> str:
        """Геттер для номера телефону."""
        return self.__phone_number

    @phone_number.setter
    def phone_number(self, new_number: str) -> None:
        """Сеттер для номера телефону з базовою валідацією."""
        if len(new_number) >= 10:
            self.__phone_number = new_number
        else:
            raise ValueError("Некоректний номер телефону.")

    def add_to_cart(self, product: Product, quantity: int) -> None:
        """Додавання товару до кошика."""
        self.cart[product] = self.cart.get(product, 0) + quantity

    def __str__(self) -> str:
        return f"Покупець [{self.customer_id}]: {self.first_name} {self.last_name}"

    def __repr__(self) -> str:
        return (f"Customer(customer_id={self.customer_id},
first_name='{self.first_name}', "
                f"last_name='{self.last_name}',
phone_number='{self.__phone_number}')")


class Order:
    LOG_FILE = "shortage_log.txt"

    def __init__(self, customer: Customer) -> None:
        self.customer: Customer = customer
        self.ordered_items: dict[Product, float] = {}
        self.total_amount: float = 0.0

    @classmethod
    def check_total_quantity(cls, cart: dict[Product, int]) -> bool:
        """Класовий метод для перевірки, чи не перевищує загальна кількість товарів
1000."""
        return sum(cart.values()) <= 1000

    def _log_shortage(self, product: Product) -> None:
        """Приватний метод для логування нестачі товару."""
        message = (f"Покупець: {self.customer.last_name}
{self.customer.first_name}, "
                   f"Тел: {self.customer.phone_number} | "
```

```
                    f"Нестача товару: {product.name} (Замовлено:
{self.customer.cart[product]}, "
                    f"В наявності: {product.quantity_in_stock})\n")

        with open(self.LOG_FILE, "a", encoding="utf-8") as file:
            file.write(message)

    @receipt_decorator
    def process_order(self) -> None:
        """Обробка замовлення з урахуванням наявності та знижок."""
        if not self.check_total_quantity(self.customer.cart):
            print("Помилка: Кількість одиниць у замовленні перевищує допустимий
ліміт (1000).")
            return

        valid_products = []

        # Перевірка наявності товарів
        for product, quantity in list(self.customer.cart.items()):
            if product.quantity_in_stock >= quantity:
                valid_products.append(product)
            else:
                self._log_shortage(product)
                del self.customer.cart[product] # Видаляємо товар, якого не
вистачає

        if not valid_products:
            return

        # Обчислення середньої ціни товарів у замовленні
        avg_price = sum(p.price for p in valid_products) / len(valid_products)

        # Розрахунок вартості зі знижками
        for product in valid_products:
            quantity = self.customer.cart[product]
            final_price = product.price

            # Знижка 10%, якщо ціна вища за середню
            if final_price > avg_price:
                final_price *= 0.9

            cost = final_price * quantity
            self.ordered_items[product] = cost
            self.total_amount += cost

            # Списання зі складу
            product.quantity_in_stock -= quantity

        self.customer.cart.clear() # Очищаємо кошик після оформлення

    def __str__(self) -> str:
        return f"Замовлення для {self.customer.first_name}
{self.customer.last_name} на суму {self.total_amount:.2f} грн."
```

```python
    def __repr__(self) -> str:
        return f"Order(customer={repr(self.customer)},
total_amount={self.total_amount})"


if __name__ == "__main__":
    if os.path.exists(Order.LOG_FILE):
        os.remove(Order.LOG_FILE)

    # Створення товарів
    p1 = Product("Хліб", 30.0, 50)
    p2 = Product("Молоко", 45.0, 20)
    p3 = Product("Кава", 250.0, 5) # Дорогий товар (потрапить під знижку)
    p4 = Product("Сир", 150.0, 2)  # Цього товару не вистачить

    print(f"Загальна кількість товарів на складі:
{Product.calculate_total_stock([p1, p2, p3, p4])} шт.\n")

    customer1 = Customer(1, "Іван", "Іваненко", "+380501234567")
    print(f"Початковий телефон: {customer1.phone_number}")
    customer1.phone_number = "+380999876543" # Використання сеттера

    # Додавання до кошика
    customer1.add_to_cart(p1, 2)   # Хліб
    customer1.add_to_cart(p2, 1)   # Молоко
    customer1.add_to_cart(p3, 1)   # Кава
    customer1.add_to_cart(p4, 5)   # Сир (нестача: просимо 5, є 2)

    # Оформлення замовлення
    order1 = Order(customer1)
    order1.process_order()

    print("Магічні методи (__str__ та __repr__):")
    print(str(p1))
    print(repr(customer1))
    print(str(order1))
```